

Assignment: The Royal Expedition Pack

Python Programming - Week 10

1. The Story

The King is sending an explorer on a dangerous mission to the Frozen Peaks of Algorithmia. The explorer has a sturdy backpack, but it can only carry a maximum weight capacity of W .

The Royal Vault has provided a list of N survival items. Each item has a specific **Weight** and a **Value** (representing its importance to the mission). As the Royal Court Coder, you must decide which items the explorer should take to maximize the total value without exceeding the backpack's weight limit.

2. Problem Statement

Given N items, each with a weight w_i and a value v_i , and a total capacity W , solve the **0/1 Knapsack Problem**.

To prove your logic is correct, the King requires:

1. The **Maximum Total Value** that can be carried.
2. The **Indices** of the items selected (1-indexed, in ascending order).

3. Technical Constraints

- $1 \leq N \leq 100$ (Number of items).
- $1 \leq W \leq 1000$ (Weight capacity).
- $1 \leq w_i, v_i \leq 500$ (Individual item attributes).
- Time Complexity: $O(N \times W)$.
- Space Complexity: $O(N \times W)$ to store the DP table for backtracking.

4. Input & Output Format

Input:

1. Line 1: Integer N (number of items).
2. Line 2: Integer W (maximum capacity).

3. Next N lines: Two space-separated integers representing the Weight and Value of item i .

Output:

- Line 1: The maximum total value.
- Line 2: Space-separated indices of the items included (1-indexed).

Example: Input:

- $N = 4$
- $W = 10$
- Item 1: Weight 5, Value 40
- Item 2: Weight 4, Value 40
- Item 3: Weight 6, Value 50
- Item 4: Weight 3, Value 30

Input:

```
4
10
5 40
4 40
6 50
3 30
```

Output:

```
90
2 3
```

(Explanation: A greedy approach might pick Item 4 first because of its low weight, or Item 2 because of its ratio. However, the optimal set is Items 2 and 3, totaling 90 value.)

5. Implementation Skeleton

Submit a file named `week10-[yourRollnumber].py`.

```
1 """
2 Name: [Your Name]
3 Roll Number: [Your Roll Number]
4 Time Complexity Analysis:
5 Explain why the complexity is  $O(N * W)$ .
6 """
7
8 def solve_knapsack(n, capacity, weights, values):
9     # Initialize DP table
10    dp = [[0 for _ in range(capacity + 1)] for _ in range(n + 1)]
11
12    # TODO: Fill the DP Table
13    # TODO: Backtrack to find the indices of items included
```

```

14
15     max_val = 0
16     selected_indices = []
17
18     return max_val, selected_indices
19
20 if __name__ == "__main__":
21     import sys
22     # Input Handling
23     try:
24         n = int(input())
25         w_limit = int(input())
26         weights = []
27         values = []
28         for _ in range(n):
29             w, v = map(int, input().split())
30             weights.append(w)
31             values.append(v)
32
33         max_val, items = solve_knapsack(n, w_limit, weights, values)
34         print(max_val)
35         print(*(sorted(items)))
36     except EOFError:
37         pass

```

6. Submission Instructions

- **File Name:** week10_[yourRollnumber].py
- **Requirement:** You must include the Time Complexity analysis in the header comment.
- **Note:** If multiple combinations yield the same maximum value, any valid combination is acceptable.